

DR.B.R.AMBEDKAR UNIVERSITY, SRIKAKULAM
COLLEGE OF ENGINEERING
Department Of Computer Science and Engineering



CERTIFICATE

This is to certify that Ms./Mr. _____ bearing the
Reg.No. _____ has satisfactorily completed the lab experiments prescribed for
Object Oriented Software Engineering Lab of B.Tech 3rd year 2nd Semester, Department of
Computer Science & Engineering Dr.B.R.Ambedkar University, Srikakulam for the academic
year 2024-25.

Signature

Faculty in Charge

Signature

Head of the Department

Examiner's signature

INDEX

S.no	Content	Page No	Remarks
1	Introduction to Software Engineering	1	
2	Introduction to Object – Oriented Software Engineering	2	
3	UML Diagrams	3	
4	A Problem Statement	13	
5	A Requirement Analysis Document	15	
6	System Requirement specification	15	
7	Project Title	16	
8	Abstract	17	
9	Hardware and Software Requirements	18	
10	UML Diagrams for “online shopping system” Application	19	
11	Testing Strategies		
12	Implementation		

13	Conclusion		
----	------------	--	--

SOFTWARE ENGINEERING

DEFINITION:

The process of manufacturing software systems. Software consists of executable computer code and the supporting documents needed to manufacture, use, and maintain the code.

Software Engineering is a modeling activity. Software engineers deal with complexity through modeling, by focusing at any one time on only the relevant details and ignoring everything else.

Software engineering is a problem-solving activity; models are used to search for an acceptable solution. This search is driven by experimentation.

Software engineering is a knowledge acquisition activity; in modeling the application and solution domain, software engineers collect data, or organize it into information and formalization into knowledge.

Software engineering is a rational-driven activity; when acquiring knowledge and making decisions about the system or its application domain.

SOFTWARE ENGINEERING FAILURES:

1. Interface Missing:

If we are developing a project for the automation of student's data for our university, finally we have to get the automation of student's data, not the other project. If we get other then it is called as interface missing.

2. User interface missing:

Every project must contain user interface otherwise it is useless.

3. High budget:

The cost of the project. Extends the appropriate cost.

4. Complexity avoidance:

The project mustn't contain bugs.

5. Time:

The project must be delivered within time otherwise the organization will loss large amount of money.

THE OBJECT-ORIENTED SOFTWARE

ENGINEERING

Object-oriented software engineering (OOSE) is an & methodology
OOSE was developed by in 1992 while at. It is the first object-oriented design methodology to employ to drive software design. It also uses other design products similar to those used by.

It was documented in the 1992 book *Object-Oriented Software Engineering: a Use Case Driven Approach*.

The tool was created by the team at Objector AB to implement the OOSE methodology. After success in the marketplace, other tool vendors also supported OOSE.

After bought Objector AB, the OOSE notation, methodology, and tools became superseded.

- As one of the primary sources of the (UML), concepts and notation from OOSE have been incorporated into UML.
- The methodology part of OOSE has since evolved into the (RUP). Rational Objector for.
- The OOSE tools have been replaced by tools supporting UML and RUP.
- OOSE has been largely replaced by the UML notation and by the methodology.

UML (UNIFIED MODELING LANGUAGE):

INTRODUCTION:

UML means Unified Modeling Language. It was developed in 1992. It is integrating with three notations.

1. OMT (Object Modeling Technique)
2. Booch
3. OOSE (Object Oriented s/w Engineering)

According to this Software engineering development activity are represented by 3 models:

1. **Functional Model:**

It represents the functionality of a system. It is represented by use case diagram.

2. **Object Model:**

It explains the structure of a system. It is represented by class diagrams, object diagrams.

3. **Dynamic Model:**

It explains the internal behavior of a system. It is represented by sequence diagrams, state chart diagrams, activity diagrams and collaboration diagrams.

TYPES OF UML DIAGRAMS

Diagrams in UML

A diagram is the graphical presentation of a set of elements, most often rendered as a connected graph of things and relationships. UML includes 9 such diagrams:

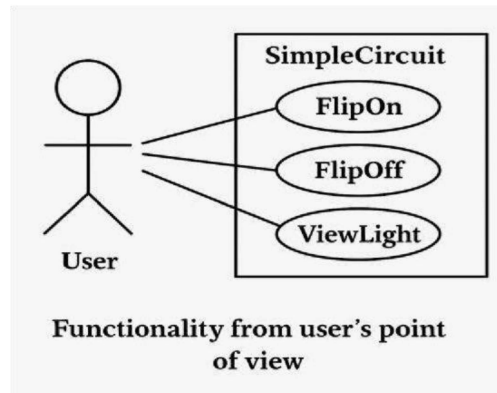
1. Class Diagram
2. Object Diagram
3. Use Case Diagram
4. Sequence Diagram
5. Collaboration Diagram
6. State Chart Diagram
7. Activity Diagram
8. Component Diagram
9. Deployment Diagram

1. Use case Diagram:

Use Case Diagrams describe the functionality of a system and users of the system. These diagrams contain the following elements:

- Actors, which represent users of a system, including human users and other systems.
- Use Cases, which represent functionality or services provided by a system to users

Example for use case diagram for a simple circuit:

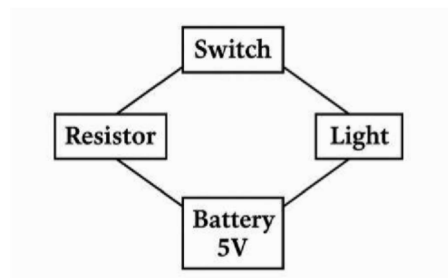


2. Class Diagrams

Class Diagrams describe the static structure of a system, or how it is structured rather than how it behaves. These diagrams contain the following elements:

- Classes, which represent entities with common characteristics or features. These features include attributes, operations and associations.
- Associations, which represent relationships that relate two or more other classes where the relationships have common characteristics or features. These attributes and operations.

Example for class diagram for a simple circuit:



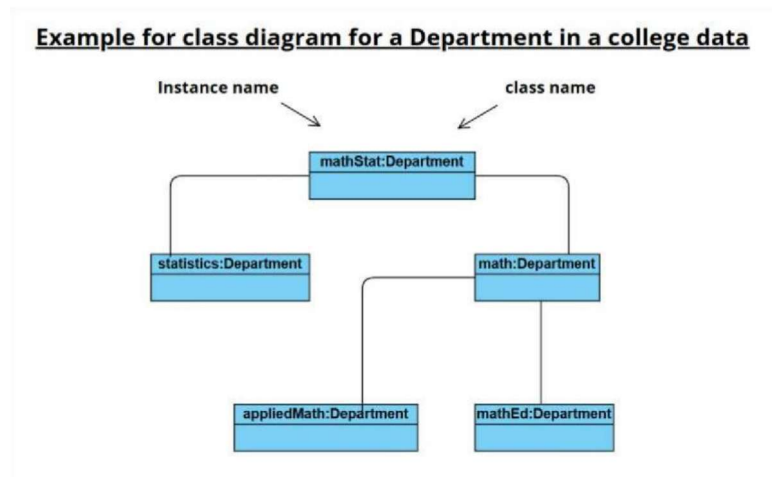
Structure of system (objects, attributes, associations, operations)

3. Object Diagrams:

Object Diagrams describe the static structure of a system at a particular time. Whereas a class model describes all possible situations, an object model describes a particular situation. Object diagrams contain the following elements:

- **Objects**, which represent particular entities. These are instances of classes.
- **Links**, which represent particular relationships between objects. These are instances of associations.

Example for class diagram for a Department in a college data:



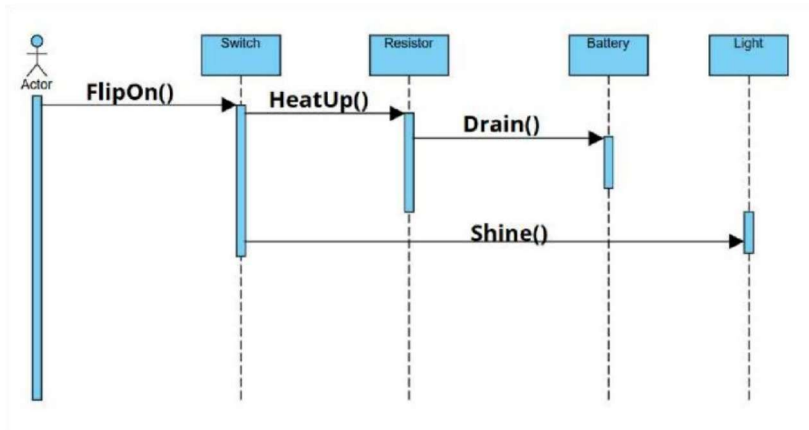
Drain: a pipe/hole in the ground that dirty water, etc. goes down to be carried away.

4. Sequence Diagram:

Sequence Diagrams describe interactions among classes. These interactions are modeled as exchange of messages. These diagrams focus on classes and the messages they exchange to accomplish some desired behavior. Sequence diagrams are a type of interaction diagrams. Sequence diagrams contain the following elements:

- Class roles, which represent roles that objects may play within the interaction.
- Lifelines, which represent the existence of an object over a period of time.
- Activations, which represent the time during which an object is performing an operation.
- Messages, which represent communication between objects.

Example for Sequence diagram for a simple circuit



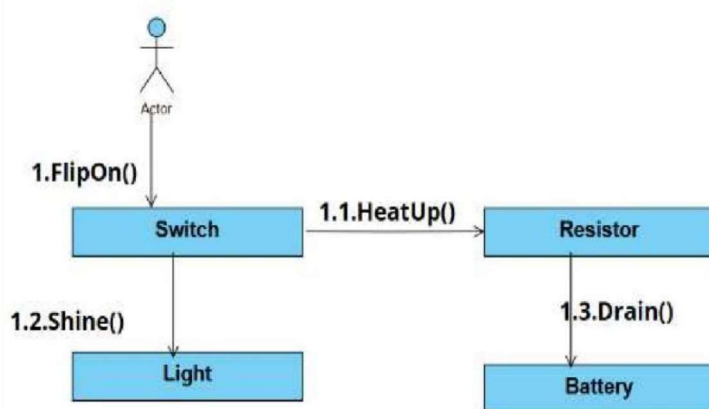
Messages between objects

5. Collaboration Diagram:

Collaboration Diagrams describe interactions among classes and associations. These interactions are modeled as exchanges of messages between classes through their associations. Collaboration diagrams are a type of interaction diagram. Collaboration diagrams contain the following elements:

- Class roles, which represent roles that objects may play within the interaction.
- Association roles, which represent roles that links may play within the interaction.
- Message flows, which represent messages sent between objects via links. Links transport or implement the delivery of the message.

Example for Collaboration Diagram for a simple circuit



More compact, but harder to interpret

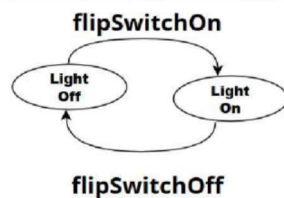
6. Statechart Diagram:

State chart (or state) diagrams describe the states and responses of a class. State chart diagrams describe the behavior of a class in response to external stimuli. These diagrams contain the following elements:

- States, which represent the situations during the life of an object in which it satisfies some condition, performs some activity, or waits for some occurrence.
- Transitions, which represent relationships between the different states of an object.

Example for state chart diagram for a simple circuit:

Example for State Diagram for a simple circuit



**Transitions between one object
(Extension of Finite State Machine(FSM) model)**

7. Activity Diagram

Activity diagrams describe the activities of a class. These diagrams are similar to state chart diagrams and use similar conventions, but activity diagrams describe the behavior of a class in response to internal processing rather than external events as in statechart diagram.

- **Swimlanes**, which represent responsibilities of one or more objects for actions within an overall activity; that is, they divide the activity states into groups and assign these groups to objects that must perform the activities.
- **Action States**, which represent atomic, or noninterruptible, actions of entities or steps in the execution of an algorithm.
- **Action flows**, which represent relationships between the different action states of an entity.
- **Object flows**, which represent the utilization of objects by action states and the influence of action states on objects.

Example for Activity Diagram for a simple circuit

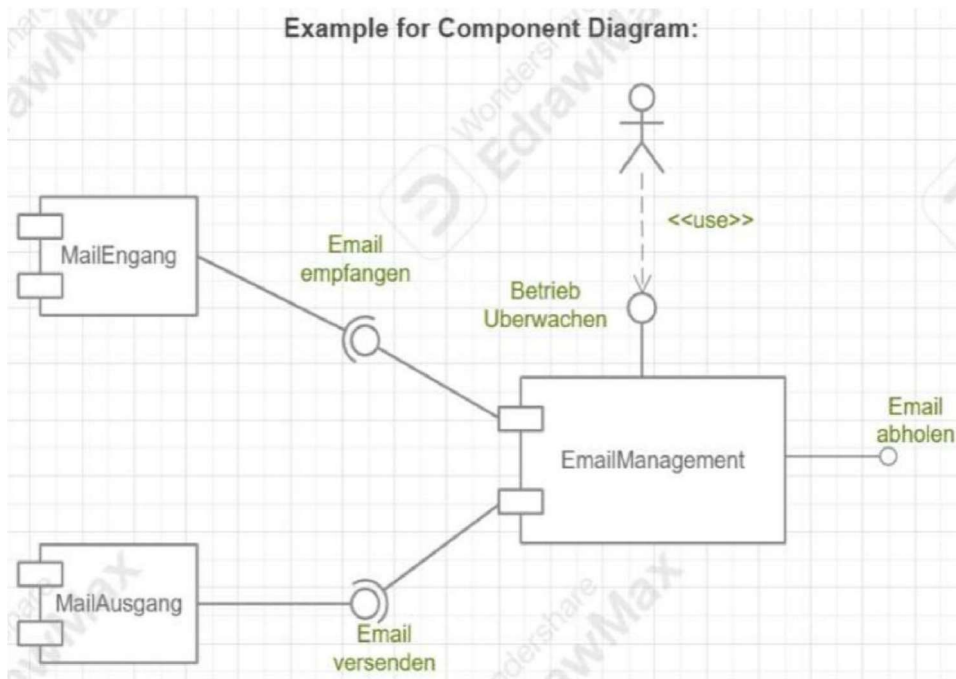


Actions are states

8. Component Diagram:

Component diagrams describe the organization of and dependencies among software implementation components. These diagrams contain components, which represent distributable physical units; including source code, object code, and executable code.

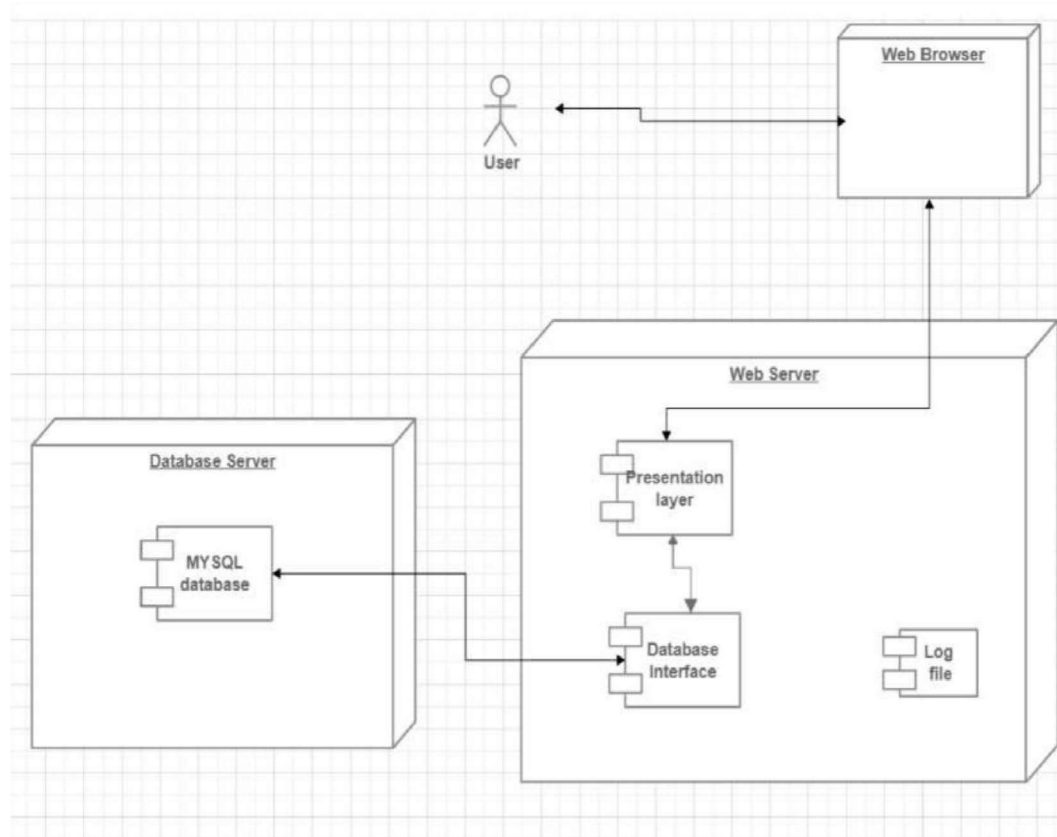
Example for Component Diagram:



9. Deployment diagrams:

Deployment diagrams describe the configuration of processing resource elements and the mapping of software implementation components onto them. These diagrams contain components and nodes, which represent processing or computational resources, including computers, printers, etc.

Example for Deployment diagram:



USE OF UML DIAGRAMS

- Software is very abstract and hard to visualize. A visual modeling language, such as UML.
- UML can be used to produce several models at increasing levels of detail.
- UML is appropriate for both new system developments and improvements to existing systems.
- The Unified Modeling Language was created by Rational Software in order to provide a standard for software modeling languages.
- The Use Case Driven nature of modeling with UML ensures that all levels of model trace back to elements of the original functional requirement

A PROBLEM STATEMENT

Aim: To prepare problem statement for any project.

Requirements:

Hardware-Interfaces:

1. Pentium (R) 4 CPU 2.26 GHz 128MB RAM
2. CDROM Driver
3. Screen Resolution of at least 800 X 600 required for proper and complete viewing of the screen.

Software-Interfaces:

Any Window page operating system (Windows 95, 98, 2000, XP, NT).

The Problem Statement is the initial starting point for a project.

It is basically 123 page statement that everyone on the project agrees with that describes what will be done at a high level.

The Problem statement is intend for a broad audiences and should be written in non-technical terms.

It helps the non-technical and technical personal communicate by providing a description of a problem.

It doesn't describe the solution to the problem.

The input to requirement engineering is the problem statement prepared by customer.

It may give an overview of the existing system along with broad expectations from the new systems.

The first phase of requirement engineering begins with requirements helisatations that is gathering of information about requirements.

Here requirements are identified with the help of customer and existing system processors.

So, from here begins the preparation of the problem statement.

So, basically a problem statement describes what needs to be done without describing how.

Conclusion: The Problem statement was written successfully by following the steps described above.

REQUIREMENT ANALYSIS DOCUMENT (RAD)

The result of requirement elicitation and the analysis activities are documented in the requirement analysis document.

This document completely describes the system in terms of functional and non-functional requirements and serves as a contractual basis between the customer and developer.

SYSTEM REQUIREMENT SPECIFICATION

It's a document that describes what a software system should do and how it should perform.

It's also called as Software Requirement document.

It's involves four elements.

-Functional Requirement:

Detailing the System behaviour and features

-Non-Functional Requirements:

Specifying the performance, security and usability criteria.

-Constraints and Assumptions:

Highlighting the limitations and pre-assumptions.

-User Interface Requirements:

Describing the system's look and feel.

Project Title

ONLINE SHOPPING SYSTEM

ABSTRACT

The online shopping system is a web-based application that allows users to buy products online easily. It helps users to browse products, add items to the cart, and place orders. The system provides secure login, payment options, and order tracking features. It also helps admin manage products and orders efficiently.

Hardware and software requirements Hardware

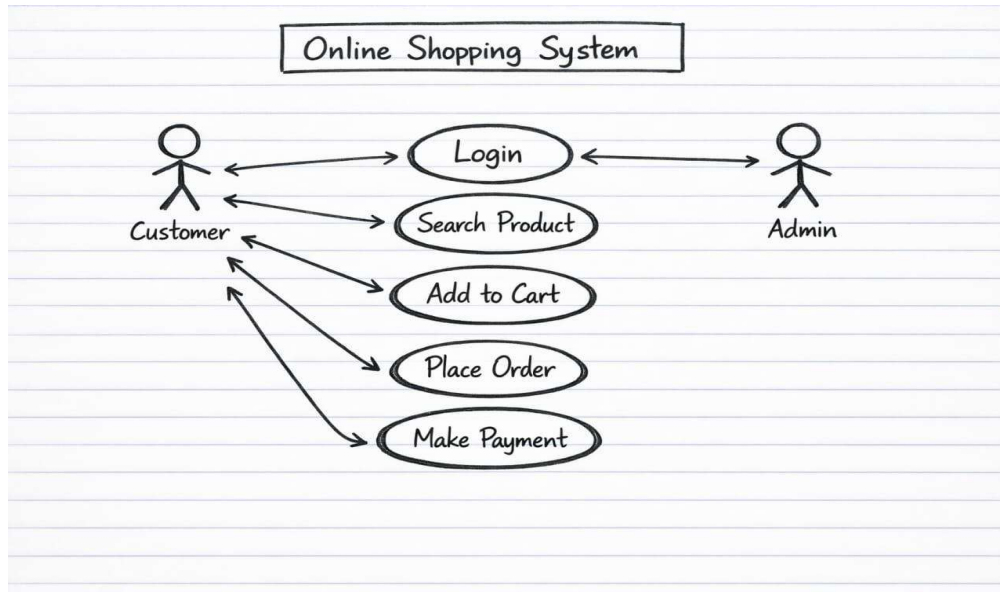
requirements:

- System Model Compaq Presario C700
- Processor Intel Pentium Dual Core T2370, 1.73 GHz:512
- Hard disk 160 GB

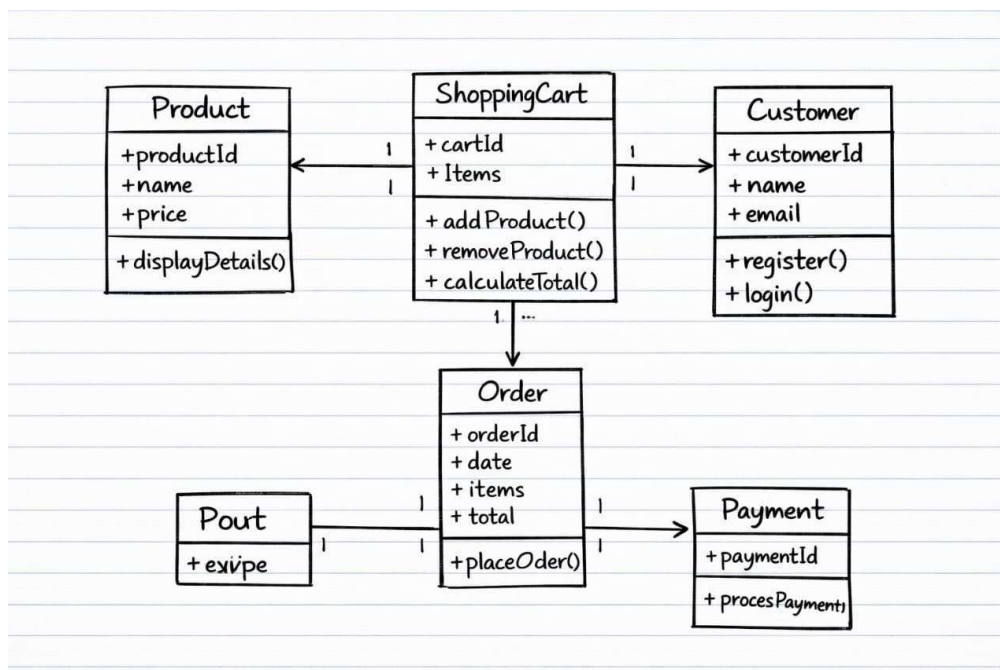
Software requirements:

- Operating System Windows XP with Service Pack 3 (CHT)
- Compiler TurboC++

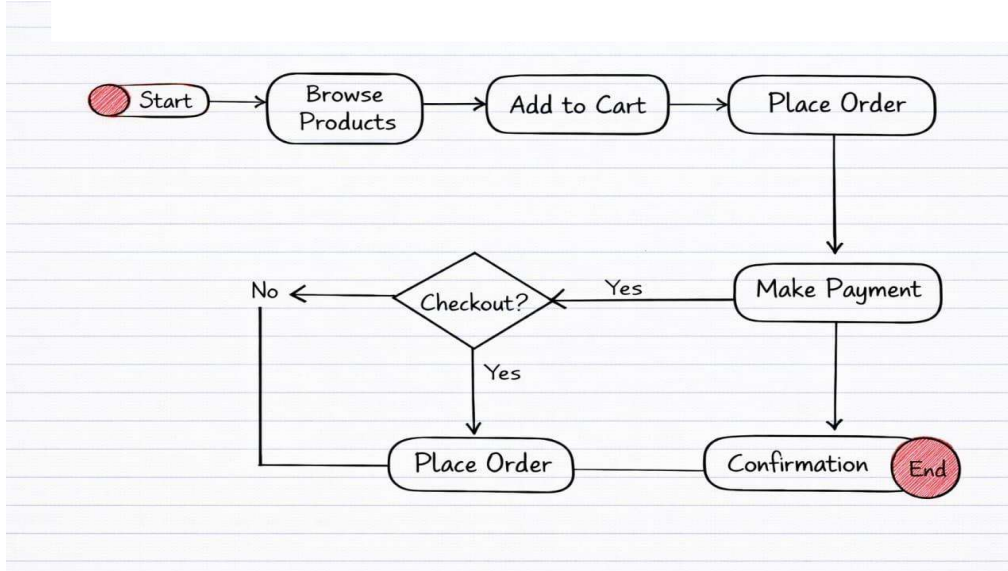
USECASE DIAGRAM



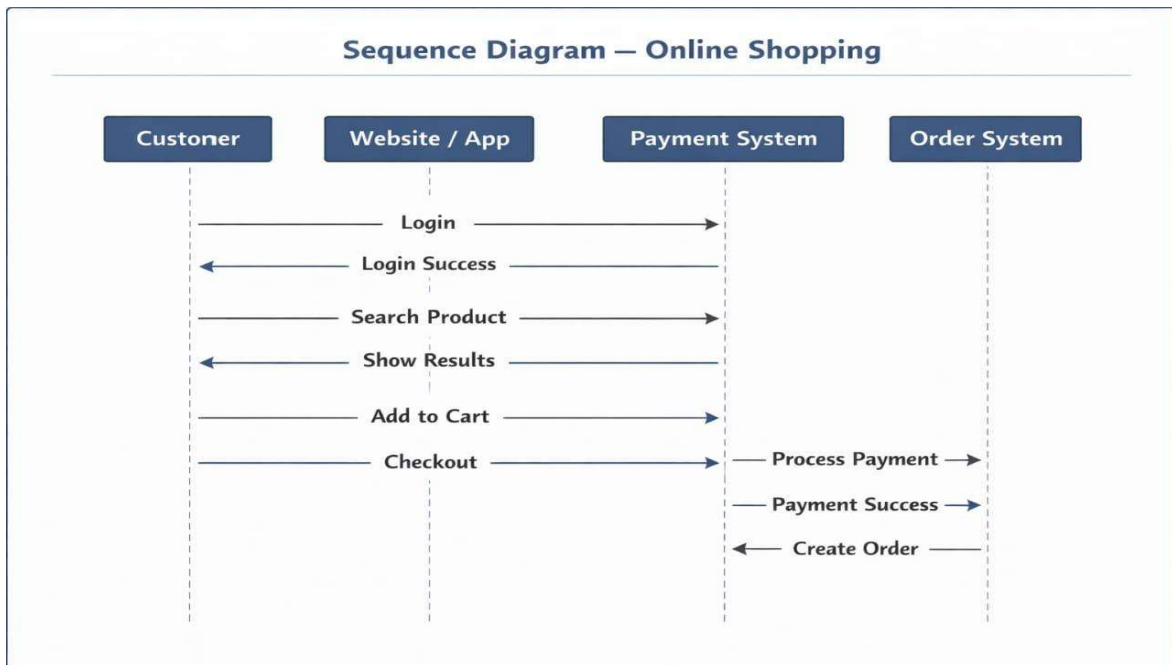
CLASS DIAGRAM



ACTIVITY DIAGRAM



SEQUENCE DIAGRAM



STATECHART DIAGRAM

State Chart — Online Shopping



Testing Strategies

Testing is a crucial phase in the software development life cycle to ensure that the system meets user requirements and performs reliably. The following testing strategies are proposed for the Railway Reservation System.

TESTING LEVELS:

Testing are frequently grouped by where they are added in the software development process, or by the level of the test.

1. Unit Testing

Objective:

To test individual modules and functions such as login, train search, reservation, cancellation, fare calculation, and database connectivity.

Tools:

- JUnit / PyTest / NUnit (based on tech stack)
- Manual testing with sample inputs

Example Scenarios:

- Verify user login with valid and invalid credentials
- Validate seat availability logic for a given train/date/class
- Test fare computation with edge cases

2. Integration Testing

Objective:

To verify data flow and interface communication between modules like Reservation ↔ Payment, Admin ↔ Train Management, etc.

Approach:

- Bottom-up integration for modules like Database → UI
- Use test stubs and drivers for intermediate modules

Key Tests:

- Reservation form → Payment gateway integration
- Admin changes reflected in passenger-facing interfaces
- Confirmation email/success page triggered after payment

3. System Testing

Objective:

To evaluate the end-to-end functionality of the system in a simulated real-world environment.

Environment Setup:

- Full deployment on test server
- Database seeded with dummy train, user, and fare data

Scenarios:

- Complete ticket booking from login to confirmation
- Booking cancellation with refund check
- Adding/modifying trains by admin and verifying impact

4. User Acceptance Testing (UAT)

Objective:

To validate the system against user expectations and ensure that it is ready for deployment.

Participants:

- End users (passengers, admins)

Focus Areas:

- Ease of navigation and booking
- Correctness of information (train names, timings, fares)
- Response time for queries and transactions

5. Regression Testing

Objective:

To ensure that new updates or bug fixes don't break existing functionality.

Approach:

- Maintain automated test scripts for major workflows
- Run full test suite after every update

6. Performance Testing

Objective:

To test system stability and speed under load.

Types:

- Load Testing: Simulate multiple users booking simultaneously
- Stress Testing: Push system beyond its limits
- Scalability Testing: Assess system behavior as data volume increases

Tools:

- Apache JMeter
- LoadRunner (optional)

7. Security Testing

Objective:

To ensure protection of sensitive data and prevent unauthorized access.

Areas to Test:

- Login and session management
- SQL injection and XSS vulnerabilities
- Admin authentication and access control

Tools:

- OWASP ZAP
- Manual ethical hacking

8. Compatibility Testing

Objective:

To verify system compatibility across different devices and browsers.

Devices/Browsers:

- Chrome, Firefox, Edge
- Android/iOS (if mobile accessible)

Test Case Description	Expected Output	Actual Output	Result
User logs in with correct username and password	User is redirected to the dashboard and greeted with a welcome message	User successfully logged in and dashboard is displayed	Pass
Booking a ticket with all valid inputs (train, passenger, payment)	Booking is successful; a confirmation page with ticket number is displayed	Booking confirmed and ticket with unique ID is generated	Pass
Searching for trains between two valid stations	A list of all available trains with timings and seat availability is shown	Train list displayed correctly with availability and fare details	Pass
Booking a ticket without selecting travel date	Error message displayed: "Please select a travel date".	System prompts the user to select a travel date	Pass
Cancelling a booked ticket	Ticket status changes to "Cancelled" and user receives confirmation message	Cancellation successful and message shown	Pass
Attempting login with wrong password	Error message: "Invalid username or password"	System displays error and does not allow access	Pass

IMPLEMENTATION

The online shopping system is developed to provide a digital platform for customers to browse and purchase products easily. The system includes secure user registration and login, product search, cart management, online payment features. Customers can view product details, add items to the cart, place orders, and track their purchases. The system is build using frontend technologies(like html,css,javascript),backend processing,and database integration to store user and product data. Overall,the system provides an efficient,reliable,and user friendly shopping experience similar to real world e-commarce platforms.

CONCLUSION

The Online Shopping System successfully simplifies the process of buying and selling products online. It reduces manual effort by automating tasks like product management, order processing, and payment handling. The system provides a convenient and time-saving solution for customers while enabling administrators to manage operations effectively. By using modular design and database management, the system ensures scalability, security, and performance. It supports digital business growth and enhances customer satisfaction through easy access, quick transactions, and real-time updates. Thus, the project demonstrates how technology can improve modern shopping experiences.